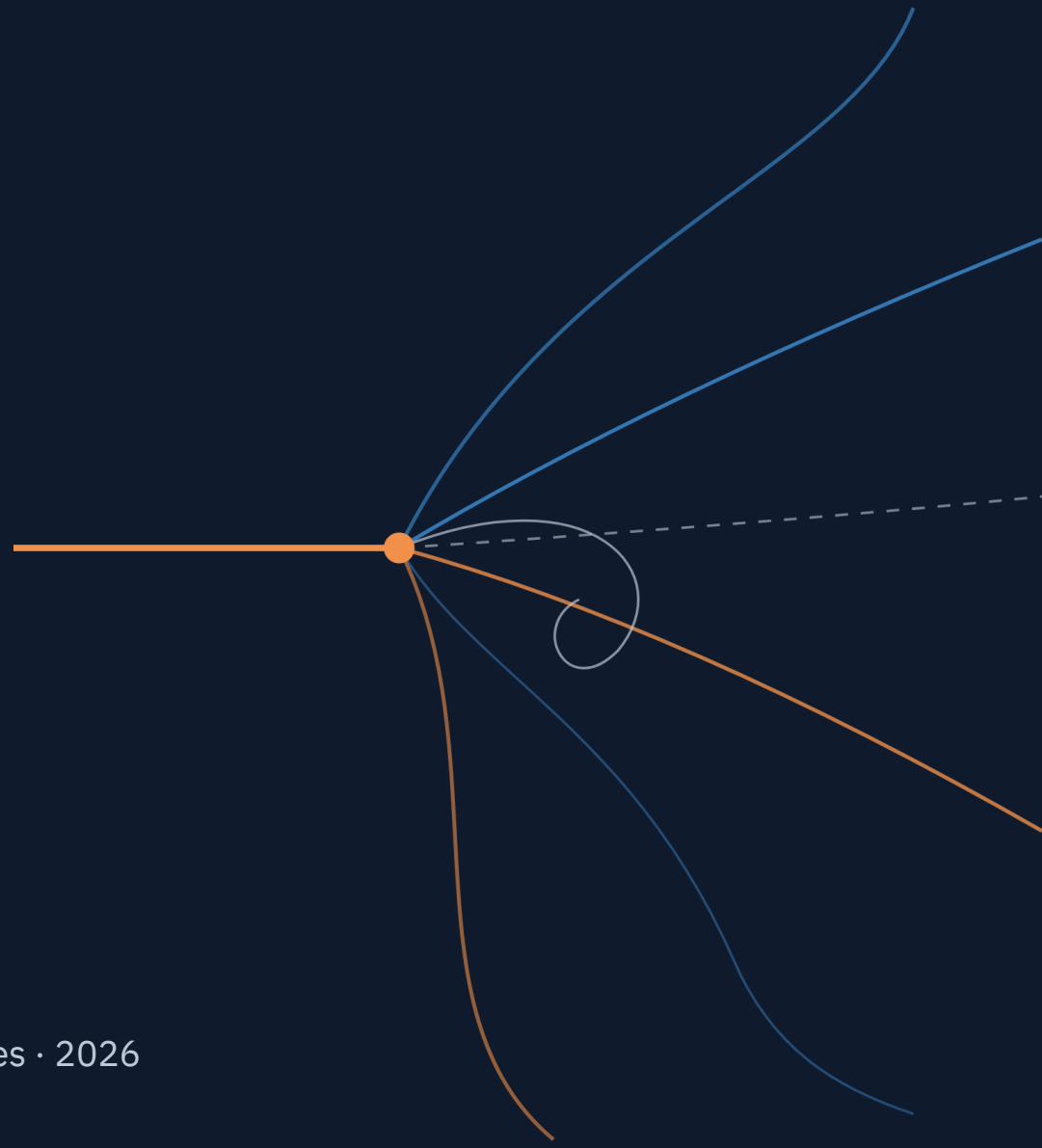
PARTICLE DETECTOR DESIGN AND SIMULATIONS

Introduction to Geant4

Simulation, the Monte Carlo Method and Geant4



Kaan Yüksel OYULMAZ · Ankara University Institute of Accelerator Technologies · 2026

Contents

01

What is simulation?

Definition, why it is used and the main types of simulation

02

The Monte Carlo method

Random sampling, the die example and particle transport

03

What is Geant4?

Applications, key concepts and the structure of an application

Simulation is imitating the behavior of a real system on a computer through a model.

Model

The system's geometry, materials and physical laws are defined mathematically.

Numerical experiment

The model is run on a computer; the conditions can be changed as often as needed.

Analysis

The outputs are interpreted and compared with real measurements.

SIMULATION

Why do we simulate?

Cost and time

We test the design virtually before building the detector.

Safety

We study dangerous conditions, such as high radiation, without risk.

Optimization

We can scan materials, thicknesses and geometry parametrically.

Interpretation

We compare measurements with the expected signal and compute efficiency and acceptance.

SIMULATION

Two basic approaches and current examples

Deterministic

Same input → always the same output

Epidemic modeling

SIR differential-equation models used during COVID-19

Engineering

Finite elements with COMSOL / ANSYS: detector cooling, mechanical strength

Space

Orbit calculation for the James Webb Space Telescope

Stochastic

Geant4 is here

Involves randomness, every run is different

Particle physics

Designing ATLAS/CMS and HL-LHC detector upgrades with Geant4

Medical physics

Proton therapy and PET dose calculations with TOPAS / GATE

Weather forecasting

ECMWF weather forecast: 50 perturbed runs

PART 2 · THE MONTE CARLO METHOD

Monte Carlo: repeated random sampling used to obtain numerical results

1777

Buffon's needle

Showed that π can be estimated by dropping needles at random.

1930s

Enrico Fermi

Computed neutron diffusion in Rome by random sampling by hand.

1946

Ulam and von Neumann

An idea born from solitaire odds was applied to neutron transport at Los Alamos.

1948

ENIAC

The first computerized Monte Carlo calculations were performed.

1953

Metropolis algorithm

Laid the foundation of Markov chain Monte Carlo (MCMC).

1974–98

GEANT → Geant4

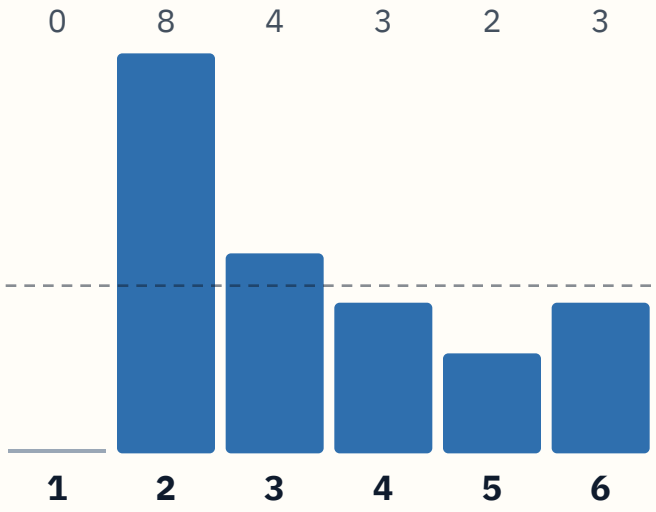
Became the standard for detector simulation at CERN.

> **Where does the name come from?** Metropolis's idea: a nod to Ulam's uncle's love of Monaco's Monte Carlo casino.

MONTE CARLO · EXPERIMENT

Which face of this die is loaded?

N = 20 rolls

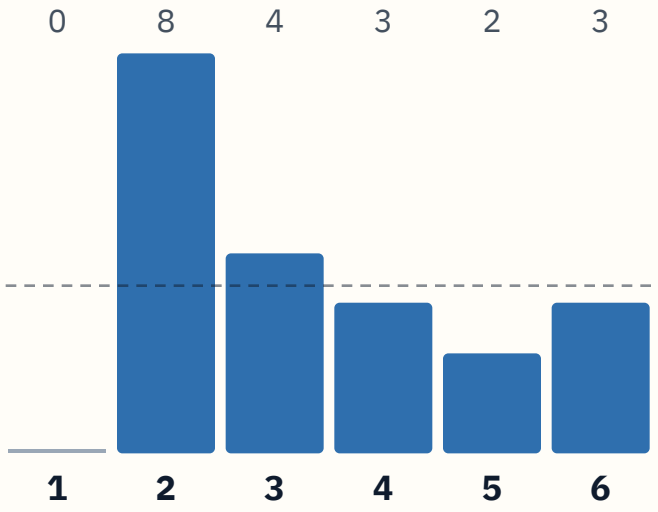


Most frequent: **2** (40%)

MONTE CARLO · EXPERIMENT

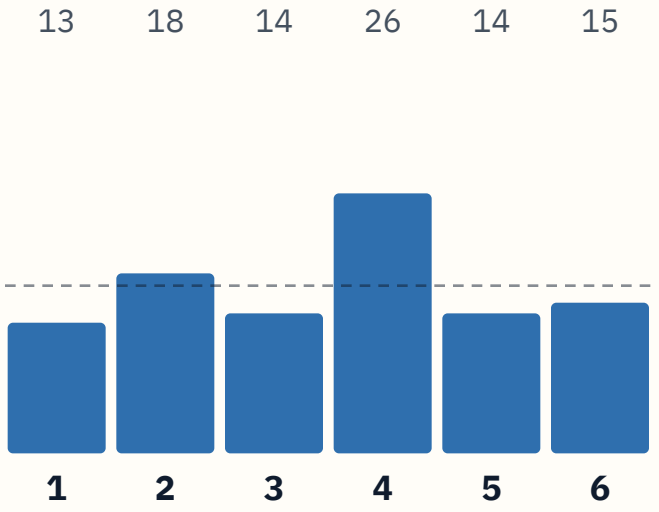
Which face of this die is loaded?

N = 20 rolls



Most frequent: **2** (40%)

N = 100 rolls

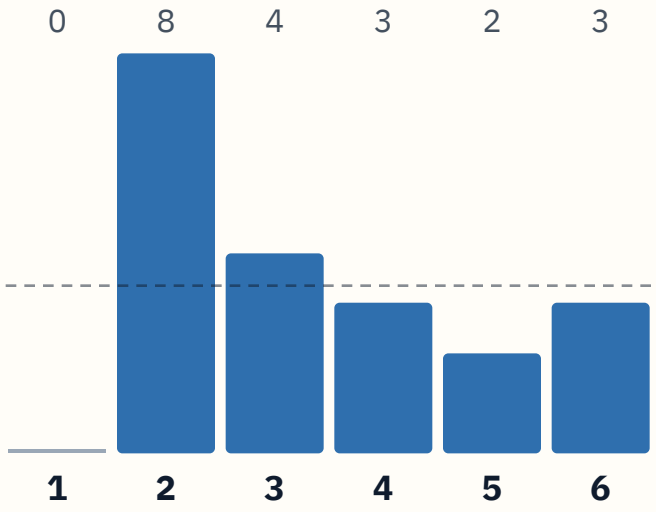


Most frequent: **4** (26%)

MONTE CARLO · EXPERIMENT

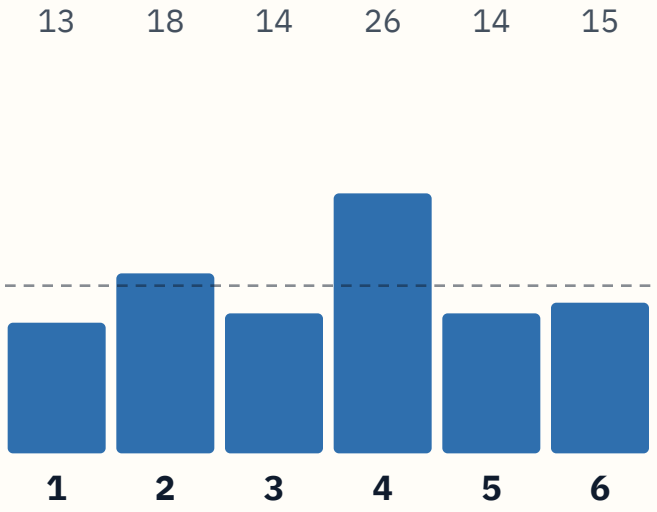
Which face of this die is loaded?

N = 20 rolls



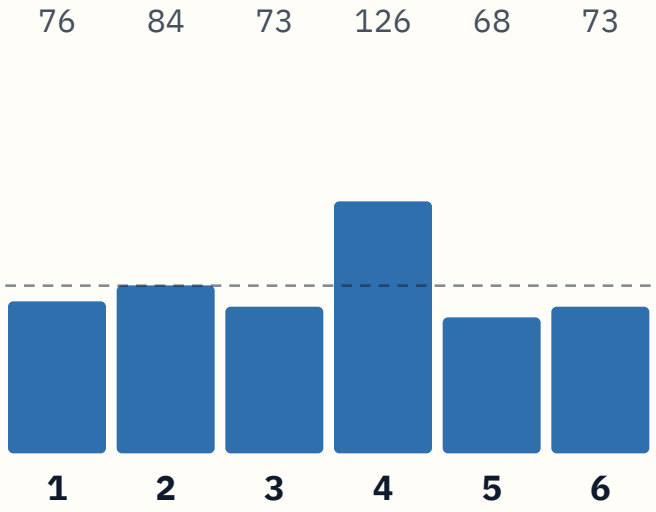
Most frequent: **2** (40%)

N = 100 rolls



Most frequent: **4** (26%)

N = 500 rolls



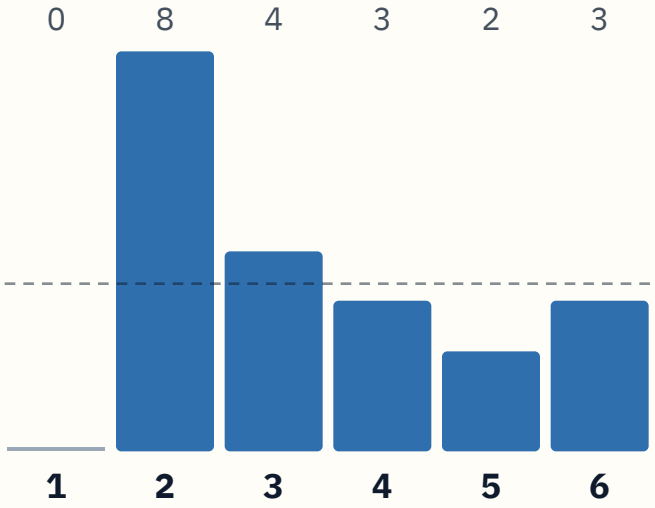
Most frequent: **4** (25.2%)

MONTE CARLO · EXPERIMENT

Which face of this die is loaded?

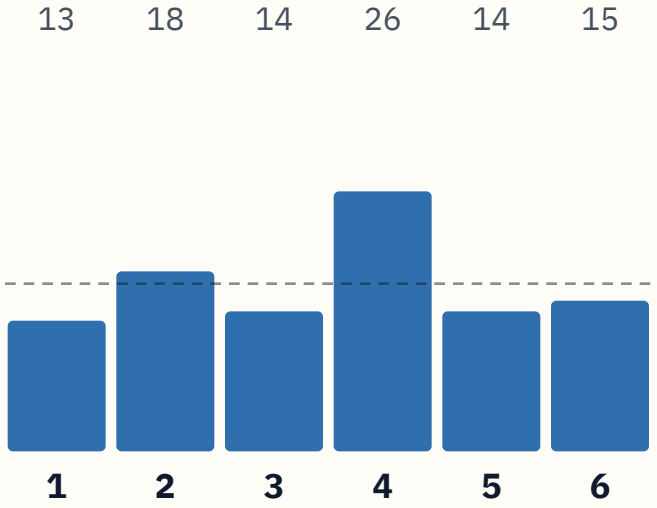
Answer: 4 is loaded.
P(4) = 0.25 · other faces 0.15 · fair: 1/6 ≈ 0.167

N = 20 rolls



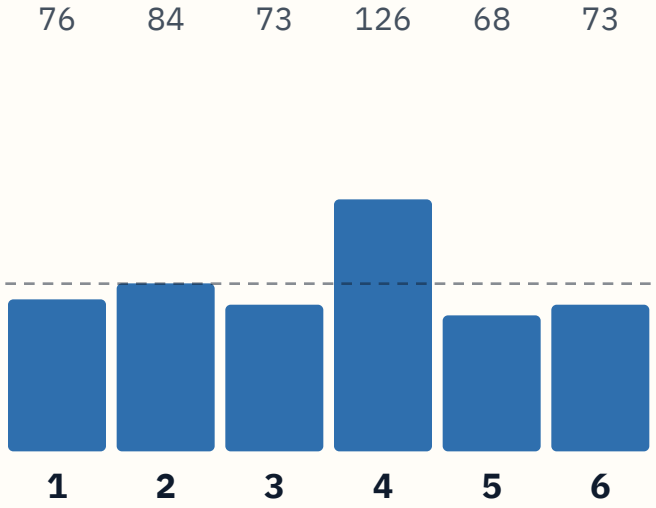
Most frequent: **2** (40%)

N = 100 rolls



Most frequent: **4** (26%)

N = 500 rolls



Most frequent: **4** (25.2%)

MONTE CARLO · THE MATH BEHIND THE EXPERIMENT

How did we find the loaded face?

1 · Roll the die

Draw $\xi \in [0, 1)$
Fair: face = $\lfloor 6\xi \rfloor + 1$
Loaded: $\xi \geq 0.75 \rightarrow 4$
else 1,2,3,5,6 (0.15 each)

2 · Measure the fraction

$\hat{p}_k = N_k / N$
$p_0 = 1/6 \approx 0.167$

3 · Measure the deviation

$\sigma_0 = \sqrt{(p_0(1 - p_0) / N)}$
$z = (\hat{p}_k - p_0) / \sigma_0$
$|z| > 3 \rightarrow$ loaded

N	σ_0	Face 2: $\hat{p}$	z	Face 4: $\hat{p}$	z	Verdict
20	0.083	0.40	2.8	0.15	−0.2	Inconclusive (z < 3)
100	0.037	0.18	0.4	0.26	2.5	4 suspicious
500	0.017	0.168	0.1	0.252	5.1	4 loaded

ξ: random number between 0 and 1
N_k: number of times face k came up
σ_0: fluctuation of the fraction for a fair die

$\lfloor x \rfloor$: round down (floor)
$\hat{p}_k$: measured fraction of face k
z: deviation in units of σ_0

N: total number of rolls
p_0: expected fraction for a fair die
$\sigma_0 \propto 1/\sqrt{N}$: shrinks as the number of rolls grows

PART 3 · WHAT IS GEANT4?

GEometry ANd Tracking

An open-source C++ toolkit that simulates the passage of particles through matter using the Monte Carlo method.

Not a ready-made program, but a **library** so the user writes their own application by defining their own geometry, physics list and particle source.

Property	Description
Developer	International Geant4 Collaboration (CERN)
Started	1994, CERN RD44 project
First release	1998
Predecessor	GEANT3 (Fortran)
Language	C++, object-oriented
License	Open source, free

GEANT4

Applications

High-energy physics

Detector simulations for LHC experiments such as ATLAS, CMS and LHCb

Medical physics

Radiotherapy, PET/SPECT imaging

Space science

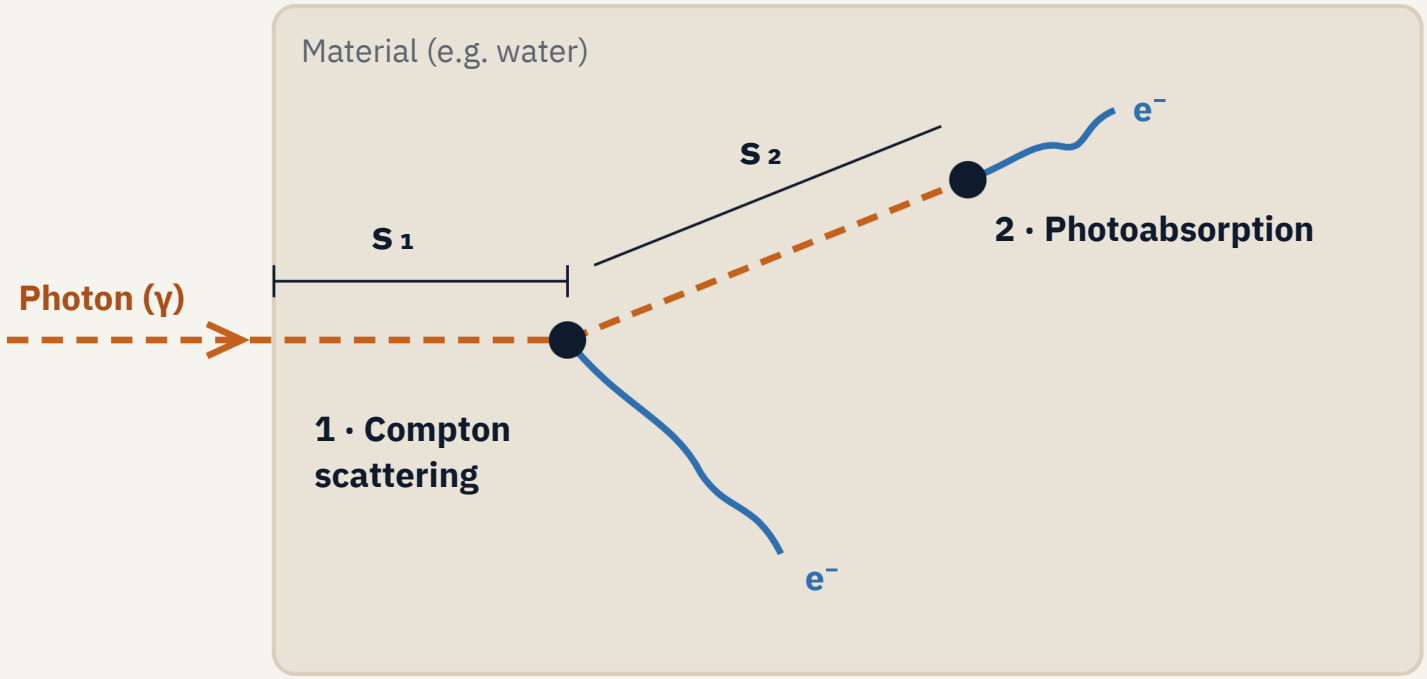
Behavior of satellite and spacecraft electronics in radiation environments

Radiation protection

Shielding calculations, dosimetry and detector response studies

GEANT4

A photon's journey through matter



① Free path

Distance to interaction: $s = -\lambda \cdot \ln \xi$

② Process selection

Roll a “loaded die” weighted by cross sections.

③ Final state

New energy and direction are sampled; secondaries (e^-) are pushed onto the stack.

④ Repeat

Until the particle is absorbed, stops or leaves the volume.

Dashed: photon · Solid: electron

GEANT4

Where does s = −λ · ln ξ come from?

Derivation

1 · Collision probability in a thin slice

$P = n \cdot \sigma \cdot dx \equiv \mu \cdot dx$

n: atoms/cm³ · σ: cross section (target area)

2 · Loss from the beam

$dN = -\mu \cdot N \cdot dx$

3 · Integrate

$\int dN/N = -\mu \int dx \rightarrow \ln(N/N_0) = -\mu x$

4 · Beer–Lambert

$N = N_0 \cdot e^{(-x/\lambda)}, \ \lambda = 1/\mu$

5 · Invert (Monte Carlo)

$\xi = e^{(-s/\lambda)} \rightarrow s = -\lambda \cdot \ln \xi$

Assumption: atoms are randomly distributed; the particle has no memory of its history.

Example: 1 MeV photon in water

1 · Molecular density

$n = \rho \cdot N_{Atoms/Mol}$

$= (1 \times 6.022 \times 10^{23}) / 18.02$

$\approx 3.34 \times 10^{22}\ cm^{-3}$

2 · Cross section (Compton)

$\sigma \approx 10\ e^- \times 0.211\ b = 2.11\ b$

3 · Attenuation coefficient

$\mu = n \cdot \sigma \approx 0.0706\ cm^{-1}$

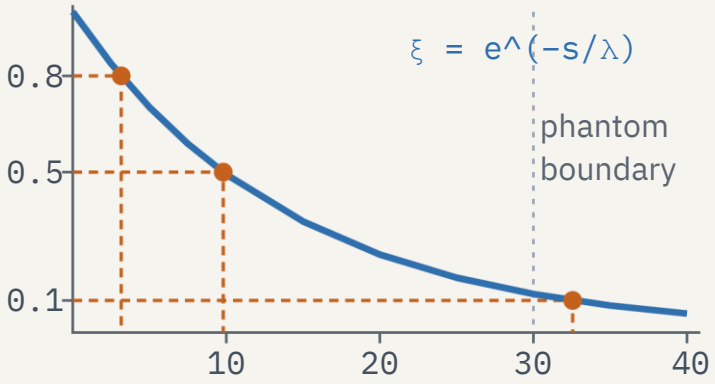
(NIST: $0.0707\ cm^{-1}$)

4 · Mean free path

$\lambda = 1/\mu \approx 14.1\ cm$

$P(10\ cm) \approx 49\% \cdot P(30\ cm) \approx 12\%$

Random ξ → path s



ξ	s (cm)	In a 30 cm phantom
0.80	3.2	interacts
0.50	9.8	interacts
0.10	32.6	escapes

GEANT4

Key concepts: Run → Event → Track → Step



Run

A set of events simulated under the same geometry and physics conditions (BeamOn).

Event

Processing of the primary particle(s) and all the secondaries they produce.

Track

The current state of a single particle: position, momentum, energy.

Step

The smallest unit, between two points; physics processes are applied and energy loss is read out here.

GEANT4

Structure of a Geant4 application

Mandatory classes

Geometry

`G4VUserDetectorConstruction`

Volumes, materials and sensitive detectors

Physics list

`G4VUserPhysicsList`

Particles and processes; reference lists: FTFP_BERT, QGSP_BIC…

Primary generator

`G4VUserPrimaryGeneratorAction`

Particle type, energy, position and direction

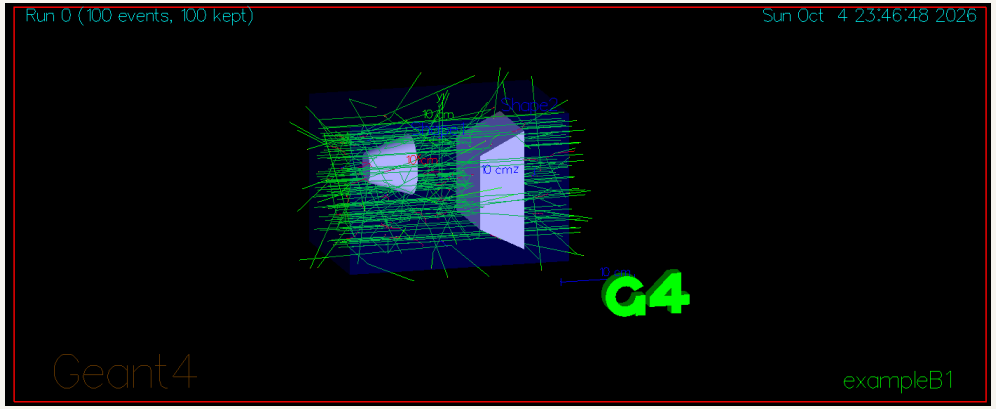
Optional user actions (registered via G4VUserActionInitialization)

`RunAction` `EventAction` `TrackingAction` `SteppingAction` `StackingAction`

GEANT4

Running an example: basic/B1

```
# 1 · Set up the environment and copy the example
$ source <install>/bin/geant4.sh
$ cp -r <geant4>/examples/basic/B1 .
# 2 · Build
$ mkdir B1-build && cd B1-build
$ cmake ../B1 && make -j4
# 3 · Run interactively
$ ./exampleB1
Idle> /run/beamOn 10
Idle> /gun/particle proton
Idle> /gun/energy 150 MeV
Idle> /run/beamOn 100
# 4 · Batch mode (with a macro file)
$ ./exampleB1 run1.mac
```



- World: air · Envelope: water
- Inner cone: A-150 tissue
- Inner trapezoid: bone, dose is scored here
- Source: 6 MeV γ · Physics list: QBBC

Output: Total dose (Gy) deposited in Shape 2 and its rms at the end of each run

CLOSING

Summary and resources

Simulation

We experiment on a real system virtually, through a model.

Monte Carlo

Solves probabilistic processes by random sampling — with error $\propto 1/\sqrt{N}$.

Geant4

Open-source C++ Monte Carlo toolkit for particle–matter interactions.

Where to start?

- geant4.web.cern.ch
- Book For Application Developers
- Examples: examples/basic/B1–B5
- Forum: geant4-forum.web.cern.ch